

Complete Presentation Script (30-40 Minutes)

Slide 1 – Introduction

"Good evening everyone.

Today, I'll be demonstrating how I built an end-to-end e-commerce application on AWS and integrated it with Dynatrace for complete observability.

Instead of explaining each component separately, I'll follow a single user request so it's easier to understand how everything works together.

During this session, I'll explain:

- how OneAgent collects metrics, logs, traces, and events,
- how that telemetry is sent to the Dynatrace platform,
- how Davis AI analyzes the data and identifies the root cause of issues,
- and finally, how dashboards, alerts, and SLOs help SRE teams detect and resolve problems quickly.

Let's get started."

-

Slide 2 – Agenda for Today

"Before we jump into the demo, let me quickly walk you through today's agenda.

The session is divided into six parts.

First, I'll give a brief introduction to **what Dynatrace is** and why modern applications need **full-stack observability**, not just traditional monitoring.

Second, we'll discuss the difference between **monitoring and observability**.

Third, we'll take a deep dive into the **Dynatrace architecture**. I'll explain the role of **OneAgent**, **ActiveGate**, the **Dynatrace SaaS platform**, and **Davis AI**, and how these components work together.

Fourth, we'll see how **OneAgent collects data automatically**. I'll explain how it captures metrics, logs, traces, process information,

Fifth, we'll follow a complete **data collection flow**. We'll trace a single user request from the browser, through dns, web server, application, and database, and see how that information reaches Dynatrace for analysis.

Finally, I'll show a live demonstration of the application that I built and deployed on AWS. We'll look at the dashboards I created, monitor real-time traffic, review alerts, and see how SLI and SLO help us measure application reliability and user experience.

So that's the roadmap for today's session. Let's begin with the first topic: What is Dynatrace?"

This slide should feel conversational. Don't read the slide word by word. Tell a story.

Slide 3 – What is Dynatrace?

"Now let's understand what Dynatrace actually is.

Dynatrace is an AI-powered observability platform.

Its job is to monitor everything that's running in our environment—from the infrastructure all the way up to the application that the customer is using.

For example, in our e-commerce application, Dynatrace continuously monitors:

- our EC2 server,
- Docker containers,
- the Python application,
- the SQLite database,
- network communication,
- and finally, the end user's experience.

Instead of opening five or six different tools, Dynatrace brings everything into **one unified dashboard**, which is why people often call it a **single pane of glass**.

Now imagine this situation.

A customer visits our website and tries to place an order.

Suddenly the page becomes slow.

As an SRE or DevOps engineer, the first questions that come to our mind are exactly the questions shown on this slide.

- **Is my application healthy?**
- **Is the database responding properly?**
- **Which service is causing the issue?**
- **Are users seeing errors right now?**

Without an observability platform, answering these questions takes time.

We might SSH into the server...

check CPU...

check memory...

look at application logs...

look at database logs...

check Docker...

and finally try to identify the root cause.

During a production incident, that investigation can easily take **30 minutes or even an hour**.

This is where Dynatrace helps.

Instead of manually checking everything, Dynatrace automatically collects all the telemetry data in real time.

It correlates the infrastructure, application, logs, traces, and user requests together.

Then **Davis AI** analyzes that data and points us directly to the root cause.

So rather than asking,

"Where should I start looking?"

Dynatrace already tells us,

"The slowdown is caused by this application process, this database query, or this infrastructure issue."

That saves a huge amount of troubleshooting time.

Slide 4 – Monitoring vs Observability

Slide 5 – Three Pillars

Metrics: Metrics tell us what is happening.

Examples: CPU Memory Disk Latency Traffic Error Rate

Logs: Logs tell us exactly what happened.

Examples: Exception, Timeout, Authentication failure, Database connection failed

Traces: Traces tell us where the delay happened.

Slide 6 – Dynatrace Architecture Overview

Transcript

"Now let's understand the architecture of Dynatrace and how the different components work together."

Dynatrace architecture is actually very simple. It has five main components, and each component has a specific responsibility.

The data always flows from top to bottom."

1. Your Infrastructure

"At the very top is **your infrastructure**.

This represents everything that you want Dynatrace to monitor.

Depending on the organization, this infrastructure can include:

- Physical servers
- Virtual machines
- Kubernetes clusters
- Docker containers
- Cloud services
- Applications
- Databases
- Network devices

In simple words, this is the environment where your business applications are running."

2. OneAgent

"The next component is **OneAgent**, which is the core component of Dynatrace.

OneAgent automatically discovers everything running on the host.

It continuously collects telemetry data such as:

- Infrastructure metrics like CPU, Memory, Disk and Network
- Application metrics
- Process information
- Service dependencies
- Logs
- Distributed traces

Once installed, OneAgent automatically starts discovering the environment."

3. ActiveGate

"After collecting the data, OneAgent sends it to **ActiveGate**.

Think of ActiveGate as a secure gateway.

Its main responsibilities are:

- Receiving telemetry from OneAgents
- Acting as a proxy for secure communication

ActiveGate securely forwards that data to the Dynatrace platform using **HTTPS over port 443**,

4. Dynatrace SaaS

"The next component is **Dynatrace SaaS**.

This is the central platform where all telemetry data is stored and processed.

Everything collected by OneAgent and reaches the Dynatrace platform.

Here, Dynatrace builds the complete topology of the environment.

It understands which application talks to which database, which services depend on each other, and how every component is connected.

This is also where users access:

- Dashboards
 - Logs
 - Metrics
 - Distributed traces
 - Service maps
 - Problems
 - Alerts
 - SLO reports"
-

5. Davis AI

"The final component is **Davis AI**, which is Dynatrace's built-in artificial intelligence engine.

Its job is not just to collect data but to analyze it.

Davis AI correlates metrics, logs, traces, events and dependencies automatically.

Instead of showing hundreds of individual alerts, it combines related events into a single problem.

It identifies the probable root cause, determines which services are affected, and shows the overall business impact.

It will reduce troubleshooting time and help operations teams resolve incidents much faster.

Slide 7 – Dynatrace Architecture

Absolutely. Here's a **natural, presentation-style transcript**. This is how you would actually speak during a KT session—not like reading a document.

Slide – Samsung SmartStore Application Architecture

"Now that we've understood how Dynatrace works, let me show you the application that I built and how Dynatrace is monitoring it end-to-end.

This is a simple e-commerce application that I built, and I'll explain the complete request flow.

Let's assume a customer wants to buy a Samsung Galaxy phone.

The journey starts:

The customer opens the website from a browser or mobile device.

The request is sent securely using **HTTPS over port 443**.

The first AWS service the request reaches is **Amazon Route 53**.

It is DNS service. it converts the domain name **vhagar.co.in** into the public IP address of my EC2 instance.

After DNS resolution, the request is forwarded to the EC2 server.

Inside AWS, I'm using an **EC2 instance** running Amazon Linux.

On this server, **Nginx** is configured as a reverse proxy.

Instead of allowing users to access the Flask application directly, all incoming requests first reach Nginx.

Nginx handles incoming client connections, performs SSL termination, and forwards the request to the application running inside Docker.

I containerized the entire application using Docker. Inside the container, I'm using **Gunicorn** as the WSGI server. (*Gunicorn is a Python application server that receives requests from Nginx and runs the Flask application.*)

Gunicorn receives the request from Nginx and manages multiple worker processes to efficiently handle concurrent user requests.

Gunicorn then forwards the request to the **Flask application**.

This is where all the business logic is implemented.

For example, when a customer views products, adds an item to the cart, or places an order, those operations are handled inside the Flask application.

Whenever the application needs product or order information, it communicates with the **SQLite database**.

The database stores product details, customer information, and order data.

Once the database returns the requested information, the response follows the same path back.

It goes from the database to Flask, from Flask to Gunicorn, then to Nginx, and finally back to the user's browser.

From the customer's perspective, the page loads within a few milliseconds.

Now let's see where **Dynatrace** comes into this architecture.

I installed **Dynatrace OneAgent** on the EC2 host.

OneAgent automatically discovered all these components.

It continuously collects three types of telemetry.

The first is **Metrics**, such as CPU usage, memory usage, disk utilization, network traffic, and container resource utilization.

The second is **Logs**, including application logs, operating system logs, and container logs.

The third is **Distributed Traces**.

Every user request is automatically traced from Nginx to Unicorn, then to the Flask application, and finally to the database.

This allows us to understand exactly where the request spent its time.

After collecting all this telemetry, OneAgent securely sends the data to the **Dynatrace SaaS platform**.

Dynatrace stores all the metrics, logs, and traces, and builds a complete topology of the application.

On top of that, **Davis AI** continuously analyzes the telemetry.

Instead of simply generating hundreds of alerts, Davis AI correlates all the data and identifies the actual root cause.

Finally, all this information becomes available in Dynatrace through **dashboards, service flow, distributed traces, logs, alerts, problems, and SLI/SLO dashboards**, which I'll demonstrate in a few minutes.

Before I move to the live demo, let me briefly explain the deployment pipeline shown at the bottom.

Whenever I push my code to **GitHub**, a **GitHub Actions** workflow automatically starts.

The pipeline builds the Docker image, performs the build process, pushes the image to **Amazon ECR**, then connects to the EC2 instance, pulls the latest image, stops the old container, and starts the new container.

This gives us a fully automated CI/CD pipeline without any manual deployment steps.

So this is the complete architecture of my Samsung SmartStore application.

It combines AWS services, Docker, CI/CD automation, and Dynatrace observability to provide complete visibility into the application's health and performance.

Now, let's switch to the live demo and see how all of this appears inside the Dynatrace dashboard."

Slide 13 – Key Takeaways

Transcript (2–3 minutes)

"Before we conclude, I'd like to summarize the key takeaways from today's session.

The first point is **Observability is more than Monitoring**.

Monitoring tells us **what happened**. For example, CPU usage is high or memory usage has increased. But observability helps us understand **why** it happened by combining **metrics, logs, and traces** in one platform. This makes troubleshooting much faster.

The second takeaway is **OneAgent provides zero-touch monitoring**.

Once OneAgent is installed on a host, it automatically discovers applications, processes, services, containers, databases, and network connections. In most cases, we don't need to manually configure each application, which saves a lot of operational effort.

The third point is **ActiveGate**.

ActiveGate is an optional component. It's commonly used in enterprise environments where servers don't have direct internet access or where additional security is required. It securely forwards monitoring data from OneAgent to the Dynatrace SaaS platform.

Finally, we have **Davis AI**.

Instead of engineers manually checking multiple dashboards and log files, Davis AI automatically correlates metrics, logs, traces, events, and dependencies. It identifies the probable root cause of an issue, helping SRE and Operations teams reduce the **Mean Time To Resolution (MTTR)** and restore services much faster.

To summarize today's demo, we built an end-to-end e-commerce application on AWS, integrated it with Dynatrace, and saw how OneAgent automatically collected telemetry data. We also explored dashboards, service monitoring, problem detection, alerts, and SLI/SLO monitoring. This demonstrates how Dynatrace provides complete end-to-end observability for modern cloud applications.

Thank you everyone for your time. I'd be happy to answer any questions."